Ejercicios Grafos

Vínculos

- Clase: Algoritmos 2 Clase 6
- Definición de clases: Clases Grafos Algoritmos
- Ejercicios: Ejercicios Grafos

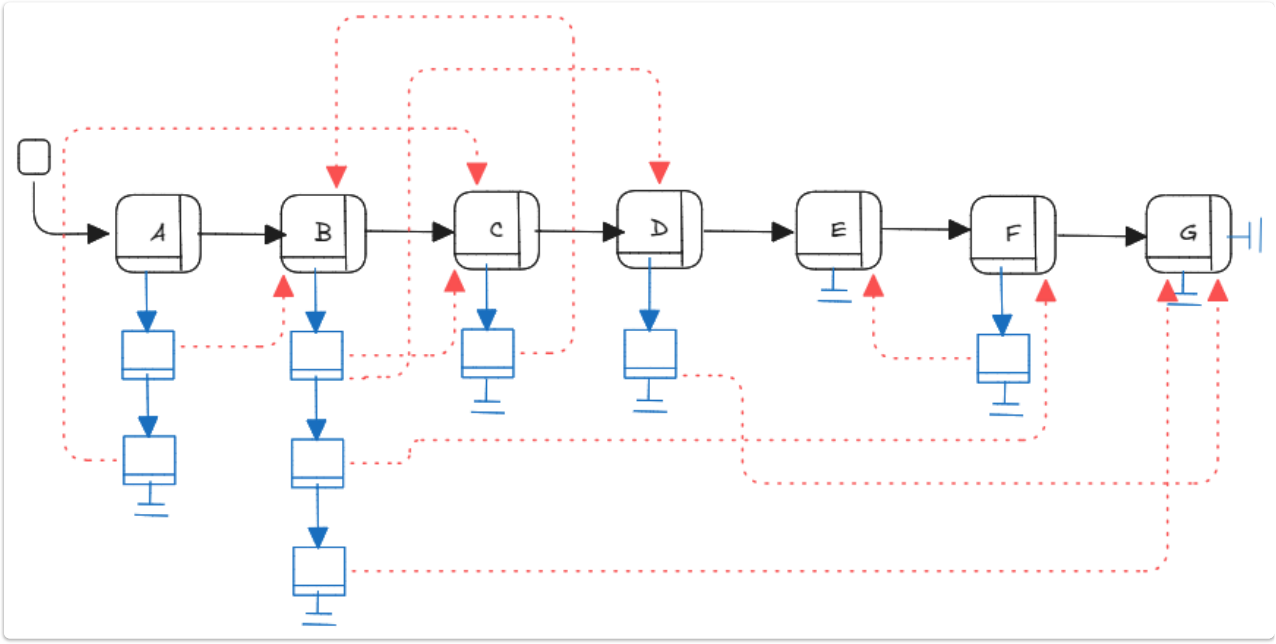
Resumen

En este documento se presentan y explican ejercicios hechos en clase y en casa del tema Estructuras Multienlazadas (Grafos), vease Algoritmos 2 Clase 6. En algunos ejercicios se utiliza inteligencia artificial para mejorar las soluciones y explicaciones de los ejercicios. A continuación la definición de las clases y la representación en memoria.

Problema	Algoritmo Recomendado	Nota Técnica
Camino más corto (no pesado)	Búsqueda en anchura	BFS
Detección de Ciclos	Búsqueda en profundidad	DFS
Conectividad	BFS o DFS	Si el recorrido visita todos los nodos.
Orden Topológico	DFS modificado	Útil para dependencias de tareas.
Laberintos	DFS	Explora hasta el fondo antes de retroceder.

> **Clases Grafos Algoritmos**
>
> ## Vínculos
>
> - Clase: Algoritmos 2 Clase 6
> - Definición de clases: Clases Grafos Algoritmos
> - Ejercicios: Ejercicios Grafos
>
> ## Representación en memoria
>
> > ### Lista de adyacencia
> >
> > Funciona con dos listas, una lista de los nodos vertices (nodos conectados linealmente) donde cada vértices tiene una lista de apuntadores a otros vértices (una lista de adyacencia).
> >
> > - Es útil en grafos no densos

- **En memoria**: $O(n+m)$ donde $n = |V| \quad m = |A|$



Clases Nodos

Nodo Arco

```
Class NodoArco
    Attributes
        Private:
            pointer to NodoVertice<Element>: info
            pointer to NodoArco<Element>: siguiente
            float: peso
    Methods
        Public:
            proc construir(...)
            // Getters
            ...
            // Setters
            ...
endClass
```

Nodo Vértice

```
Class NodoVertice
    Attributes
        Private:
            Element: info
            pointer to NodoVertice<Element>: siguiente
            pointer to NodoArco<Element>: listaAdy
    Methods
        Public:
            proc construir(Element: info, pointer to NodoVertice<Element>: sig, pointer to
NodoArco<Element>:listaAdy)
            // Getters
            ...
            // Setters
```

```
            ...
endClass
```

Clase Grafo

> *En métodos como el DFS se utiliza un método interno privado recursivo de igual nombre con diferentes parámetros (polimorfismo).*
> *De manera similar, los métodos públicos recursivos requieren un método privado, ya sea polimórfico o no.*

```
Class Grafo
    Attributes
        Private:
            pointer to NodoVertice<Element>: g
            int: nVertices, nArcos
            bool: esDirigido
    Methods
        Public:
            proc constructor(...)
            // Getters
            ...
            // Setters
            ...

            func getVertices(): Lista<Element> // O(n)
            func getArcos(): Lista<Lista<Element>> // O(n + m)
            func getPesoArco(Element: v, w): float // O(n + arcos(v))
            func getSucesores(Element: v): Lista<Element> // O(n+ arcos(v)) Solo en dirigidos
            func getPredecesores(Element: v): Lista<Element> // O(n + m)
            func getGradoVertice(Element: v): int // O(n + arcos(v))
            func getVecinos(Element: v): Lista<Element> //O(n + arcos(v)) No dirigidos
            proc insertarVertice(Element:v) // O(1) Se inserta al principio
            proc insertarArco(Element: v, w, float: peso) // O(n)
            proc eliminarVertice(Element: v) // O(n + m)
            proc eliminarArco(Element: v, w) // O(n + arcos(v) + arcos(w))
            proc setPeso(Element: v, w, float: peso) // O(n + arcos(v) + arcos(w))
            func bfs(): Lista<Element> // O(n + m)
            func dfs(): Lista<Element> // O(n + m)
endClass
```

Ejercicios

Mapeado de un grafo (en especial en programación)

- **Definición**: El mapeo es la técnica de asociar un `Element` (ej. un String "Caracas") con un índice entero único entre 1 y N.
- **Implementación**: Generalmente se usa una **Tabla de Hash** o un Diccionario (un map de C++) dentro de la clase `Grafo`.
- **Utilidad**: Permite que el arreglo de `visitados` sea de acceso usando el índice obtenido del mapeo.

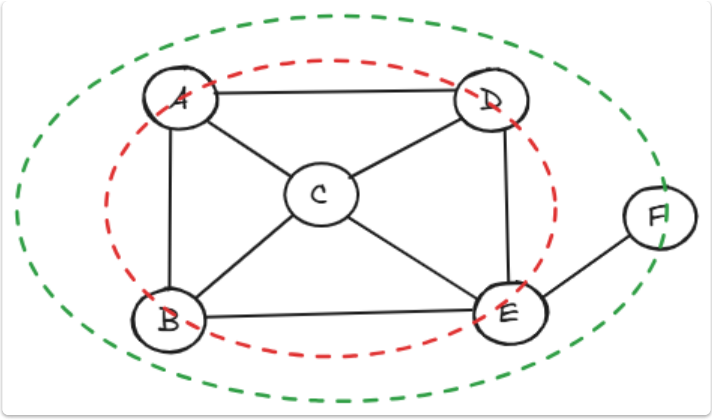
> *Se sobreentiende que en algoritmos los grafos ya están mapeados*

Implementación de Métodos Críticos

BFS (Búsqueda en anchura)

Funciona de manera parecida a un recorrido por niveles, donde se van buscando todos los vertices adyacentes al nodo, se marca como visitado y después se va iterando los vertices en busca de los demás elementos no-visitados.
Características:

1. Obtiene la componente conexa de V. (Grafo no dirigido)
2. Obtiene los nodos alcanzables. (Grafos dirigido)
3. Obtiene el camino más corto en arcos.
4. Es O(n + m)
5. Puede quedarse sin recorrer vértices no alcanzables por V.

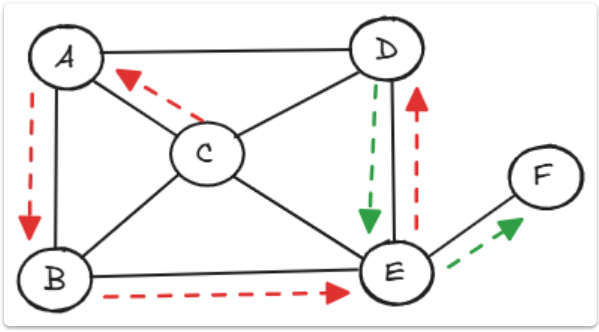



```
func BFS(Grafo<Element>: g, Element: v):
Lista<Element>
    Var
        Cola<Element>: colaAux
        array[1..N]of bool: visitados
        Lista<Elemen>: recorrido, vecinos
        int: n, i, w
    Begin
        colaAux.construir()
        recorrido.construir()
        vecinos.construir()
        n <- g.getNVertices()
        for i<-1 to N do
            visitados[i] <- false
        endfor
        colaAux.encolar(v)
        visitados[v] <- true

        while ¬colaAux.esVacia() do
            vecinos <-
g.getVecinos(colaAux.getFrente())
            while ¬vecinos.esVacia() do
                w <- vecinos.consultar(1)
                if ¬visitados[w] then
                    colaAux.encolar(w)
                    visitados[w] <- true
                endif
                vecinos.eliminar(1)
            endwhile
            recorrido.insertar(colaAux.getFrente(),
recorrido.getLong() +1)
            colaAux.desencolar()
        endwhile
        return recorrido
endfunc
```

DFS (Búsqueda en profundidad)

La búsqueda en profundidad va buscando los vértices accesibles desde el nodo inicial de manera recursiva. Cuando encuentra un nodo no-visitado lo marca como visitado y busca recursivamente los arcos con vertices no-visitados también hasta que no encuentre ninguno más.



$$Visitados = \begin{array}{|cccccc|} A & B & C & D & E & F \\ \hline 1 & 1 & 1 & 1 & 1 & 1 \end{array}$$

$$Recorrido = [C, A, B, E, D, F]$$

Cuando se recorre, los arcos se pueden clasificar. Esto es vital para detectar ciclos:

1. **Arcos de Árbol**: Arcos que pertenecen al bosque de expansión (cuando visitas un nodo por primera vez).
2. **Arcos de Retroceso (Back Edges)**: Arcos que van de un nodo a un ancestro en el árbol DFS. **¡Si existen, el grafo tiene ciclos!**

3. **Arcos de Avance (Forward Edges)**: Arcos que van de un nodo a un descendiente ya visitado.
4. **Arcos de Cruce (Cross Edges)**: Arcos entre ramas diferentes del árbol.

```
func DFS(Grafo<Element>: g, Element: v): Lista<Element>
    Var
        array[1..N] of bool: visitados
        Lista<Element>: recorrido, vecinos
        int: i, n
    Begin
        recorrido.construir()
        vecinos.construir()
        n <- g.getNVertices()
        for i <- 1 to n do
            visitados[i] <- false
        endfor
        DFS(g, v, visitados, recorrido)
        return recorrido
endfunc
proc DFS(ref Grafo<Element>: g, Element: v, ref array[1..N] of bool: visitados, ref Lista<Element>: recorrido)
    Var
        Lista<Element>: vecinos
        Element: w
    Begin
        recorrido.insertar(v, recorrido.getLong() +1)
        visitados[v] <- true
        vecinos <- g.getVecinos(v)
        while ¬vecinos.esVacia() do
            w <- vecinos.consultar(1)
            if ¬visitados[w] then
                DFS(g, w, visitados, recorrido)
            endif
            vecinos.eliminar(1)
        endwhile
endproc
```